

行业智能体项目报告

一、背景

本人在某银行软件开发中心工作，工作主要内容是银行线上系统的功能开发和运维。在日常工作中，因为人员流动和现有资料繁多，运维支持人员在不了解代码、表结构等基础信息的情况下，很多时候需要开发人员依据代码来梳理小众分支以及配合查数来应对客户特殊需求。如某小众设置要如何实现、被盗刷了需要尽可能多的支付信息（网点交易记录之外的信息）等。这些运维支持人员解决不了的具体问题最终被交到开发人员手中，这一方面意味着运维能力有待提升，一方面意味着开发人员负担加重。

在大语言模型发展的当下，一个领域精通的大模型能够很好的代替开发人员成为一本易于查询易于使用的工具书。本项目旨在实现一个原型框架，使得大语言模型能够结合知识库内特有知识，通过提示词工程实现对专业问题的解答。同时，我们提供了简易查数工具，便于大模型可从数据库获取回答需要的相关信息。

二、流程改进方案

为实现通过大语言模型解答专业领域问答且保持准确的需求，设计如下流程：

1、收集问答知识、表结构知识等一系列可能用到的专业知识和资料，构建知识库。本项目采用 json 存储，便于原型实现，实际操作中可使用知识图谱等结构型知识库以提升效果。

2、通过嵌入模型将知识条目提取为向量并保存。

3、大模型区分闲聊场景和专业场景，专业场景则检索知识库相关知识

4、大模型分析是否需要执行数据库查询操作，若需要则生成 sql，并调用外部工具执行语句。

5、大模型综合相关知识和 sql 查询结果（如果有）进行回答。

三、实现细节

本节分别展示各模块实现内容如下：

1、知识库结构和索引构建：

本项目采用 json 结构的问答对形式保存相关知识，便于实验扩展和直观展示。

```
{
  "id": 1,
  "question": "云闪付如何绑定银行卡?",
  "answer": "打开云闪付App -> 点击右下角【我的】 -> 选择【银行卡】 -> 点击【添加银行卡】 -> 输入卡号、预留手机号 -> 验证短信验证码完成绑定。"
```

通过预先下载的 paraphrase-multilingual-MiniLM-L12-v2 模型对知识向量化，并保存到对应文件中：

```
# ----- 构建索引（仅在直接运行此脚本时执行） -----
if __name__ == "__main__":
    # 1. 读取知识库
    with open(KNOWLEDGE_JSON, 'r', encoding='utf-8') as f:
        knowledge_base = json.load(f)

    # 2. 编码问题
    questions = [item['question'] for item in knowledge_base]
    question_embeddings = model.encode(questions, convert_to_numpy=True)

    # 3. 构建 FAISS 索引
    dimension = question_embeddings.shape[1]
    index = faiss.IndexFlatIP(dimension)
    faiss.normalize_L2(question_embeddings)
    index.add(question_embeddings)

    # 4. 保存索引和元数据
    faiss.write_index(index, INDEX_FILE)
    with open(METADATA_FILE, 'w', encoding='utf-8') as f:
        json.dump(knowledge_base, f, ensure_ascii=False, indent=2)

    print(f"索引构建完成，共 {len(knowledge_base)} 条记录，保存至 {INDEX_FILE} 和 {METADATA_FILE}")
```

2、通过提示词实现对输入问题的分类，判断输出结果，若为日常聊天则直接调用大模型给出返回，专业问答则走设定流程：

```
你是一个对话路由器。用户会输入一段话，你需要判断它是：
- "professional": 用户询问某个金融App（如云闪付、中国工商银行app等）的具体使用方法、功能操作、账户问题、交易异常、优惠活动等业务相关的问题。
- "casual": 用户只是日常聊天、打招呼、表达情绪、或者与金融App完全无关的闲聊。

只输出一个单词：professional 或 casual。不要输出任何其他内容。
```

3、检索可能用到的 topk 知识，作为 rag 附加给后续大模型调用：

```
def retrieve_similar_queries(user_query, top_k=3):
    """
    从知识库中检索与用户问题最相似的 top_k 条知识。
    返回列表，每个元素包含 question, answer, score
    """
    # 加载索引和知识库（实际使用中可缓存到全局变量，避免重复加载）
    index, knowledge_base = load_knowledge_base()

    # 编码用户问题
    query_vec = model.encode([user_query], convert_to_numpy=True)
    faiss.normalize_L2(query_vec)

    # 检索
    scores, indices = index.search(query_vec, top_k)

    results = []
    for idx, score in zip(indices[0], scores[0]):
        if idx != -1 and idx < len(knowledge_base):
            item = knowledge_base[idx]
            results.append({
                'question': item['question'],
                'answer': item['answer'],
                'score': float(score)
            })
    return results
```

4、根据检索结果调用大模型判断是否需要使用外部工具(此处为执行 sql 语句)：

```
你是一个路由器。用户的问题是：“{user_query}”
参考资料是：“{context}”
请判断该问题是否需要通过查询数据库解答，如果查回表结构相关知识，优先考虑通过查询数据库解答。
只回答“是”或“否”。
```

5、若需要执行 sql，则传入知识和客户提问，用于 sql 的生成：

```
根据以下表结构，将用户问题转换为 SQL 查询语句。只输出 SQL 语句，不要有其他解释。SQL 必须是只读的 SELECT 查询。

客观知识:
{context}

用户问题: {user_query}

SQL:
```

6、让大模型综合提问、知识、数据，给出相关答复。

```
{
  "role": "system", "content": "你是金融App专家。请根据以下参考知识和真实数据回答用户问题。如果知识不足以解答，请直接说明，不要编造。",
  "role": "user", "content": f"用户问题: {user_query}\n\n参考知识: \n{context}\n\n查询语句: \n{sql}\n\n结果数据: \n{select_result}"
}
```

四、项目展示

1、使用过程

执行 search.py 把知识向量化并保存。

```
(rag_env) MacBook-Air:Advanced-Artificial-Intelligence-Project yrc$ python ./knowledge_base/search.py
Loading weights: 100%
BertModel LOAD REPORT from: /Users/ycr/my_computer/code/Advanced-Artificial-Intelligence-Project/model_dir
Key | Status |
---|---|
embeddings.position_ids | UNEXPECTED |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
索引构建完成，共 4 条记录，保存至 kb_index.faiss 和 kb_metadata.json。
```

执行 main.py 开启服务。

```
(rag_env) MacBook-Air:Advanced-Artificial-Intelligence-Project yrc$ python main.py
Loading weights: 100%
BertModel LOAD REPORT from: /Users/ycr/my_computer/code/Advanced-Artificial-Intelligence-Project/model_dir
Key | Status |
---|---|
embeddings.position_ids | UNEXPECTED |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
请输入您的问题:
```

输入问题：我该如何查看我的云闪付上有什么优惠券

可以看到分类结果为：professional 专业问答，查询到两条相关知识，判断无需查询数据库，最终给出符合预期的答复。

```
请输入您的问题：我该如何查看我的云闪付上有什么优惠券
[分类结果] professional
context: [参考问题] 怎么查看云闪付的优惠券？
[解决方法] 首页点击【我的】->【我的票券】即可查看所有可用优惠券、过期券。

【参考问题】云闪付转账失败怎么办？
【解决方法】转账失败常见原因：1) 银行卡余额不足；2) 单日限额超限；3) 对方账户状态异常。建议检查余额和限额，或联系发卡行确认。
is_data_query answer is:否

[助手回答]
您好，查看云闪付优惠券的方法很简单：请打开云闪付App，在首页点击右下角的【我的】，然后进入【我的票券】页面，即可查看所有可用的优惠券以及已过期的券。如果还有其他问题，欢迎随时咨询！
```

输入问题：我怀疑我的账户被盗刷了，请帮我查看一下这张卡有没有什么异常：621226123421235

可以看到分类结果为：professional 专业问答，查询到两条相关知识，判断需要查询数据库，然后大模型依据知识生成了对应的 sql 语句，并查回结果，给出符合预期的答复。

```
请输入您的问题：我怀疑我的账户被盗刷了，请帮我查看一下这张卡有没有什么异常：621226123421235
[分类结果] professional
context: [参考问题] 卡号、支付表、支付记录、消费记录
【解决方法】查询支付表，支付表结构如下：表名：pay_table；字段清单：id-唯一索引、cardnum-卡号、amount-支付金额、date-消费日期。参考语句：select cardnum,amount,date from pay_table where cardnum = {},cardnum取自提问中卡号

【参考问题】怎么查看云闪付的优惠券？
【解决方法】首页点击【我的】->【我的票券】即可查看所有可用优惠券、过期券。
is_data_query answer is:是
generate_sql result is :SELECT cardnum, amount, date FROM pay_table WHERE cardnum = '621226123421235' ORDER BY date DESC;
[{'cardnum': '621226123421235', 'amount': Decimal('100.00'), 'date': datetime.date(2026, 4, 19)}, {'cardnum': '621226123421235', 'amount': Decimal('50.00'), 'date': datetime.date(2026, 4, 18)}, {'cardnum': '621226123421235', 'amount': Decimal('70.00'), 'date': datetime.date(2026, 4, 16)}]

[助手回答]
根据您提供的卡号 **621226123421235**，我查询了支付记录，以下是该卡近期的消费明细：

| 卡号 | 支付金额 | 消费日期 |
|---|---|---|
| 621226123421235 | 100.00元 | 2026-04-19 |
| 621226123421235 | 50.00元 | 2026-04-18 |
| 621226123421235 | 70.00元 | 2026-04-16 |

**初步分析：**
- 该卡在 **2026年4月16日至19日** 期间有3笔消费，金额分别为70元、50元、100元，均为小额交易。
- 如果您本人没有在上述日期进行这些消费，或者对其中某笔交易有疑问，建议您：
  1. **立即联系发卡银行**（如工商银行等，卡号前6位621226通常对应工行）进行挂失或冻结账户。
  2. 要求银行提供更详细的交易商户信息，以核实是否为盗刷。
  3. 检查是否有其他未显示的异常交易（如境外、大额等）。

目前仅靠支付记录无法直接判断是否被盗刷，需要您结合自己的消费记忆来判断。如有更多信息（如交易商户、时间点等），我可以进一步协助分析。
```

输入：今天天气不错，有什么好吃的推荐，分类结果为日常聊天，大模型自由回答。

请输入您的问题：今天天气不错，有什么好吃的推荐
[分类结果] casual
[助手回答]
哈哈，天气好心情也好！这时候最适合来点清爽又治愈的美食啦～比如一份芒果椰奶冻，冰冰凉凉带着热带甜香，或者来碗酸辣开胃的泰式凉拌粉丝，配杯气泡柠檬水，完美！如果想吃热乎的，试试芝士焗红薯，拉丝香甜，吃完幸福感爆棚～你更偏爱甜口还是咸口呀？🍓

输入指令 exit，结束问答。

请输入您的问题：exit
结束问答。

2、使用效果

该项目能较好的依据参考知识对相关技术问题做出较为准确的答复,可以用作运维工作时候的辅助工具作为参考,随着知识库的不断丰富和模型分支的稳定,该项目可一定程度成为一名经验丰富的专家来协助进行相关答疑。

交互速度方面,因目前流程较为简单,普通问答不超过 5s,即使最为复杂的查询数据库场景,也不超过 10s 即可得到返回,同时用户输入交互会减轻对时间的感知,整体而言可以接受。

五、总结与思考

本项目作为一个基础原型,实现了通过大语言模型来负责一定的业务场景的问答以及针对需要数据查询的场景自动生成 sql 语句进行查数,预期功能实现完整。因对应场景是针对多个运维人员交流形式,对模型反馈输出速度要求不高,工程化之后可满足基本要求。

由于是涉及客户和专业领域的知识,对回答准确性要求较高,且真实场景中数据量繁杂,远大于原型实验中涉及的专业知识,针对这个现象可考虑采用知识图谱来形成结构化知识保持其准确性,同时选取更多的相关知识,通过大模型进行一定程度的筛选,来保证能获取到更全面的知识。也可考虑针对不同具体业务场景划分具体分支,形成多个专家模型,从而实现较好的搜索效果。

近一步的实验依赖行内涉密资料和大量数据支持,若后续有机会落地,可在此基础上改进。